

Linguaggi Formali e Compilatori

(Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 2 febbraio 2010 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE

NOME:

MATRICOLA:

FIRMA:

ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche libere e automi a pila
 3. analisi sintattica e parsificatori
 4. traduzione sintattica e analisi semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie.
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.30m - Parte II (esercitazioni): 45m

1 Espressioni regolari e automi finiti 20%

1. Si considerino i due linguaggi regolari L_1 e L_2 seguenti, di alfabeto $\{a, b, c\}$:

$$L_1 = \{ a x b y \mid x \text{ e } y \text{ sono stringhe qualsiasi, } y \text{ non è vuota} \}$$

$$L_2 = \{ z \mid \text{la stringa } z \text{ non contiene il digramma (coppia) } bc \}$$

Si risponda alle domande seguenti:

- (a) Si ricavino le due espressioni regolari R_1 e R_2 che generano i linguaggi L_1 e L_2 , rispettivamente, precisando se sono ambigue o no.
 - (b) Si ricavino i due automi finiti A_1 e A_2 che riconoscono i linguaggi L_1 e L_2 , rispettivamente, precisando se sono deterministici o no.
 - (c) (facoltativa) Si ricavi l'automa finito A_3 che riconosce il linguaggio regolare intersezione $L_3 = L_1 \cap L_2$, precisando se è deterministico o no.
-

Soluzione

- (a) L'espressione regolare R_1 è piuttosto intuitiva esaminando la stringa di L_1 :

$$R_1 = a (a \mid b \mid c)^* b (a \mid b \mid c)^+$$

Naturalmente questa formulazione è ambigua, p. es. la stringa $abbb$ è generata in due modi diversi.

L'espressione regolare R_2 è lievemente più complessa:

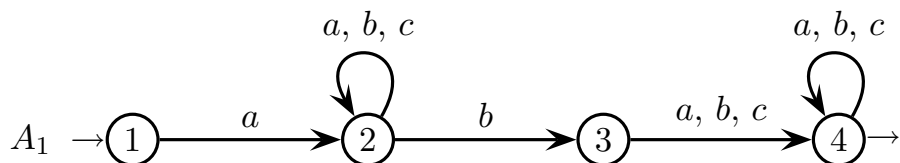
$$R_2 = (a \mid c)^* (b^+ a (a \mid c)^*)^* b^*$$

Si ottiene R_2 notando che nella stringa di L_2 la distribuzione di lettere a e c è del tutto libera, ma che ogni lettera b isolata od ogni fattore $b \dots b$ costituito solo da lettere b , dev'essere seguito da una lettera a , non da una lettera c , a meno di trovarsi in fondo alla stringa. Una variante, più compatta ma un poco meno esplicita, è la seguente:

$$R'_2 = (a \mid c \mid b^+ a)^* b^*$$

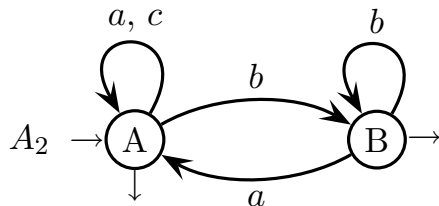
Si noti che R_2 non è ambigua (e neppure R'_2), perché le lettere b interne, isolate o consecutive e sempre seguite da una lettera a , sono evidentemente generate in modo unico dalla sottoespressione $b^+ a$, e quelle eventuali finali sono pure generate in modo unico dalla sottoespressione b^* ; e ragionamento analogo vale per le lettere a e c iniziali o interne.

- (b) Naturalmente si possono trasformare le espressioni regolari R_1 e R_2 in automi seguendo uno dei procedimenti algoritmici noti. Qui si procede intuitivamente, data la semplicità del compito. Ecco l'automa A_1 :



Si noti che l'automa A_1 è indeterministico nello stato 2, a causa degli archi b multipli.

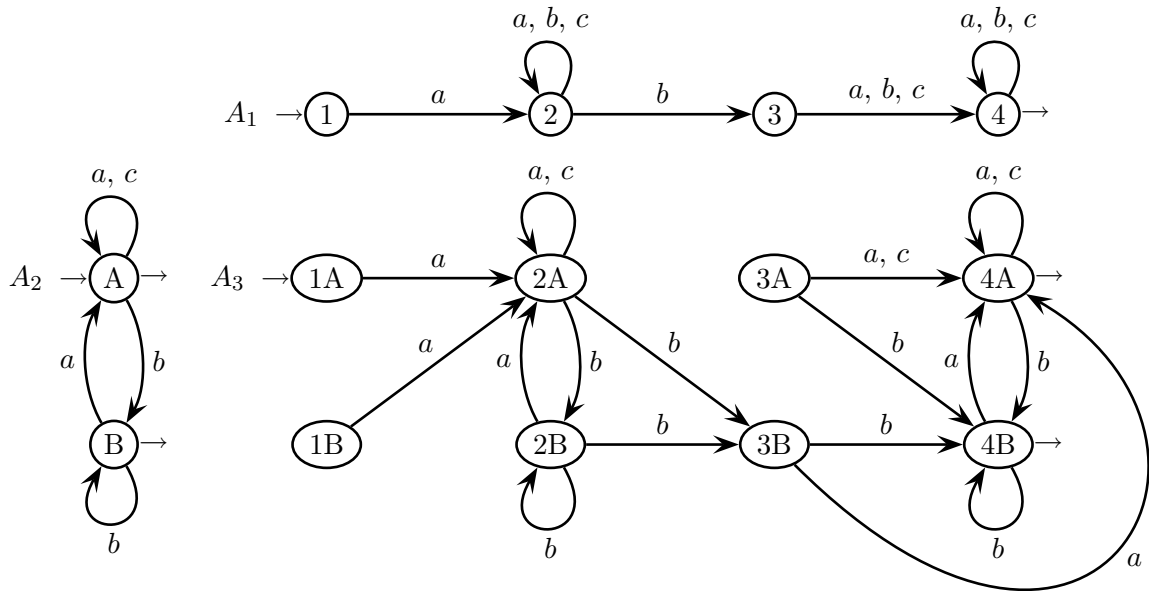
L'automa A_2 è ottenibile intuitivamente ispirandosi all'espressione regolare R_2 . Eccolo:



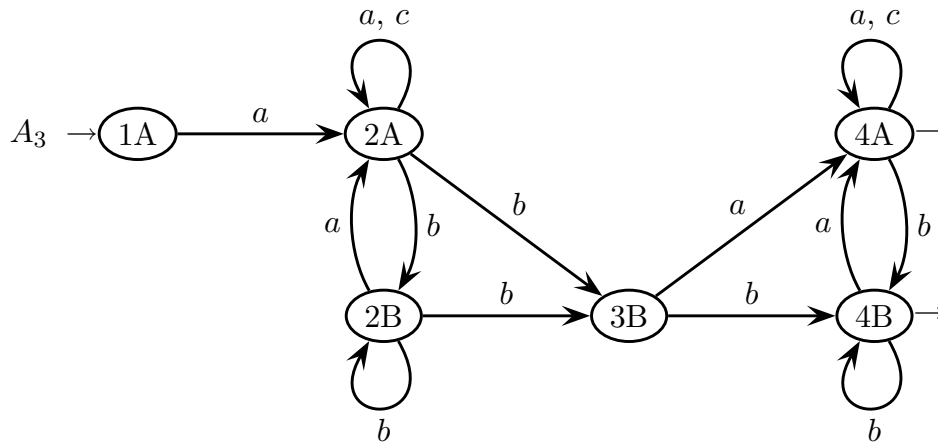
Si noti che l'automa A_2 è deterministico ed evidentemente anche minimo.

- (c) Ora per trovare l'automa intersezione A_3 , si può costruire il prodotto cartesiano d'automi $A_1 \times A_2$. Per costruzione A_3 ha $4 \times 2 = 8$ stati. Tuttavia alcuni stati

possono essere inutili (irraggiungibili o indefiniti) e vanno eliminati, riducendo l'automa. Ecco la costruzione di A_3 :



Gli stati $1B$ e $3A$ sono irraggiungibili e pertanto eliminabili. L'automa A_3 si riduce così, con solo 6 stati:



Dopo la riduzione l'automa A_3 è indeterministico negli stati $2A$ e $2B$, a causa degli archi b multipli. Beninteso, volendo lo si potrebbe determinizzare ed eventualmente minimizzare.

2. È dato il linguaggio regolare L di alfabeto $\{a, -\}$ definito dalla e.r. seguente:

$$L = a (- a)^+$$

Si consideri il linguaggio L_{err} ottenuto introducendo in ogni frase di L *uno, e un solo, errore*, che può essere del tipo seguente:

- sostituzione di una lettera ‘ a ’ con il segno ‘ $-$ ’
- sostituzione di un segno ‘ $-$ ’ con la lettera ‘ a ’

Ecco alcuni esempi di stringhe del linguaggio L_{err} :

stringa $\in L_{err}$	proveniente da
$- - a$	$a - a$
$a a a$	$a - a$
$a - -$	$a - a$
$- - a - a$	$a - a - a$
$\dots$	$\dots$
$a - a - -$	$a - a - a$

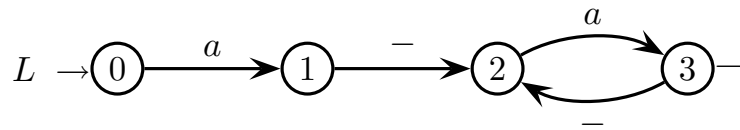
Nota bene: le stringhe di L , ossia quelle prive d'errore, *non* appartengono a L_{err} .

Si risponda alle domande seguenti:

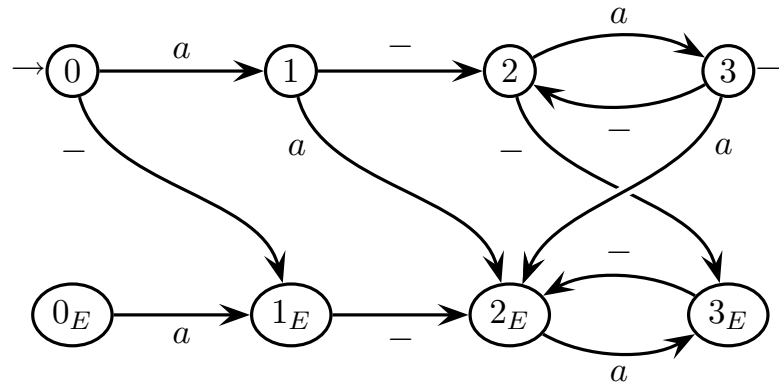
- Facendo comprendere il procedimento seguito, si costruisca un automa finito deterministico A che riconosce il linguaggio L_{err} .
- (facoltativa) Si calcoli l'espressione regolare R del linguaggio riflesso L_{err}^R .

Soluzione

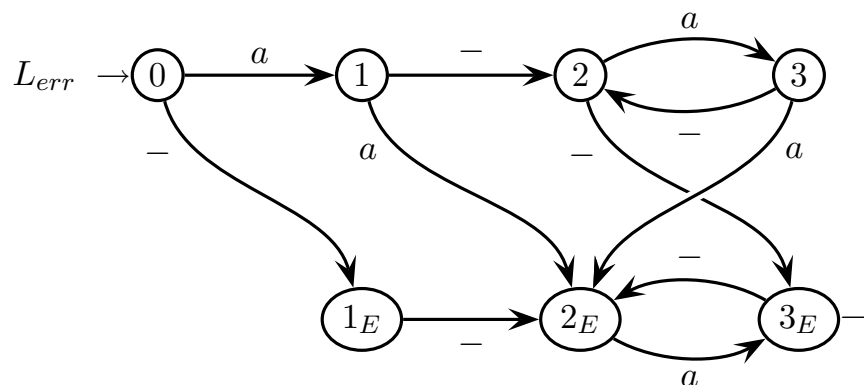
(a) Convieni disegnare l'automa del linguaggio L :



Ecco ora una seconda copia dell'automa, contrassegnandone gli stati con la lettera E . Da ogni stato dell'automa originale ha origine un arco che legge 'a' oppure '-'. Se in tale stato interviene un errore, lo stato raggiunto è quello marcato E , corrispondente allo stato che verrebbe raggiunto dalla transizione senza errore. Si ottiene così l'automa seguente:



Infine si elimina lo stato irraggiungibile 0_E , dunque inutile, e si sposta la freccia finale dallo stato 3 allo stato 3_E . Ecco l'automa di L_{err} :



L'automa così ottenuto è deterministico.

- (b) Il linguaggio L consiste di espressioni aritmetiche di addizione, senza parentesi. Dato che l'addizione è commutativa, il linguaggio L^R , riflesso del linguaggio L senza errore, è evidentemente uguale a L . Similmente, poiché l'errore commuta una lettera in un segno o viceversa, il linguaggio L_{err}^R , riflesso del linguaggio L_{err} con un solo errore, è uguale a L_{err} . Pertanto l'espressione regolare R di L_{err}^R coincide con quella di L_{err} e la si ottiene agevolmente da quella di L , così:

$$R = - (-a)^+ \mid a (-a)^* a a (-a)^* \mid a (-a)^* - - (-a)^*$$

Essa consta di tre termini uniti, che introducono uno e un solo errore o nella lettera iniziale o in una lettera successiva oppure in un segno. Volendo, si può raccogliere a fattore comune e semplificare così:

$$R = - (-a)^+ \mid a (-a)^* (a a \mid - -) (-a)^*$$

unificando il secondo e terzo termine.

2 Grammatiche libere e automi a pila 20%

1. Si consideri una variante del linguaggio di Dyck dove ci sono tre tipi di parentesi: graffe, quadre e tonde. Le stringhe del linguaggio soddisfano le regole usuali delle espressioni parentetiche, e in aggiunta le seguenti:

- le coppie di parentesi più esterne sono graffe
- le parentesi allo stesso livello di annidamento sono del medesimo tipo
- una coppia di parentesi graffe può contenere, al livello di annidamento subito sotto, solo parentesi quadre; similmente una coppia di quadre può contenere, al livello subito sotto, solo tonde; e una di tonde, subito sotto, solo graffe; si procede così circolarmente e la profondità di annidamento è arbitraria

Ecco alcuni esempi:

$$\{ \} \quad \{ [] \} \quad \{ [] \} \{ \} \quad \{ [] [] \} \quad \{ [] [()] \}$$

$$\{ [(\{ \})] \}$$

Ed ecco alcuni controesempi:

$$[] \quad \{ \{ \} \} \quad \{ () \} \quad \{ [] () \} \quad \{ [([])] \}$$

Si risponda alle domande seguenti:

- (a) Si scriva la grammatica G_{circ} del linguaggio così descritto.
- (b) (facoltativa) Si consideri una variante del linguaggio precedente, dove valgono le prime due regole enunciate e i tipi di parentesi si susseguono in modo circolare come descritto sopra, dall'esterno verso l'interno. Ma se a un certo livello la regola di annidamento circolare è violata, per esempio perché una coppia di parentesi graffe ne contiene immediatamente una di tonde, allora l'annidamento può proseguire solo con parentesi dello stesso tipo; riprendendo l'esempio, tonde immediatamente annidate in una coppia di graffe possono contenere solo altre tonde. Si scriva una grammatica G_{var} anche per questo secondo linguaggio.

Soluzione

(a) Ecco la grammatica G_{circ} (assioma G):

$$G_{circ} \left\{ \begin{array}{l} G \rightarrow \{ Q \} G \mid \varepsilon \\ Q \rightarrow [T] Q \mid \varepsilon \\ T \rightarrow (G) T \mid \varepsilon \end{array} \right.$$

La grammatica G_{circ} è costruita partendo dalla nota formulazione di Dyck, non ambigua. L'idea della costruzione è che i nonterminali G , Q e T generano coppie di parentesi graffe, quadre e tonde, rispettivamente, e che gli annidamenti di coppie procedono circolarmente come spiegato sopra, mentre i concatenamenti mantengono il tipo di parentesi corrente. La grammatica G_{circ} non è ambigua.

(b) Ed ecco la variante G_{var} (assioma G):

$$G_{var} \left\{ \begin{array}{l} G \rightarrow \{ Q \} G \mid \{ G' \} G \mid \{ T' \} G \mid \varepsilon \\ Q \rightarrow [T] Q \mid [Q'] Q \mid [G'] Q \mid \varepsilon \\ T \rightarrow (G) T \mid (T') T \mid (Q') T \mid \varepsilon \\ G' \rightarrow \{ G' \} G' \mid \{ \} \\ Q' \rightarrow [Q'] Q' \mid [] \\ T' \rightarrow (T') T' \mid () \end{array} \right.$$

La costruzione è simile alla precedente, ma in aggiunta sono espressi i vari casi di violazione nell'annidamento circolare tra coppie di parentesi. Essi sono resi dai nonterminali G' , Q' e T' , per le coppie errate graffe, quadre e tonde, rispettivamente. Le regole che espandono questi tre nonterminali aggiuntivi sono quelle usuali di Dyck (esclusa la stringa vuota), ma dove il tipo di coppia, sia annidata sia concatenata, è fisso. Neppure la grammatica G_{var} è ambigua.

2. Si consideri un linguaggio di programmazione ispirato a SETL, che definisce e opera su variabili di tipo insieme. Tale linguaggio deve comprendere le caratteristiche seguenti:

- gli elementi degli insiemi sono lettere maiuscole e minuscole, e numeri interi
- tipi e variabili sono liste di elementi, non vuote, come

```
{ 'a', 'A', 'z' }  
{ 'a', 'A', '-1', 'b', '34' }  
...
```

- le parole chiave **alphabet** e **integer** specificano il tipo insieme alfabeto (lettere maiuscole e minuscole) e intero (numeri positivi e negativi), rispettivamente
- l'identificatore di tipo o variabile è definito come in linguaggio C
- il tipo insieme va dichiarato così:

```
typedef alpha1 over alphabet;  
typedef alpha2 over { 'a', 'A', 'z' };  
typedef beta1 over { 'a', 'A', '-1', 'b', '34' };  
typedef alpha_num over alpha1 intersect beta1 unite { 'C', '9' };  
...
```

dove: **typedef** e **over** sono parole chiave; **unite** e **intersect** sono operatori che indicano unione e intersezione d'insiemi, rispettivamente; l'intersezione ha precedenza sull'unione; e nell'espressione *sono vietate* parentesi

- la variabile insieme va dichiarata così:

```
integer var1;  
alpha_num set2, set3;  
...
```

proprio come in linguaggio C

- lo statement è un assegnamento di un'espressione a una variabile, come

```
set2 = ( var1 unite { 'a', '1', 'D', '+5' } ) intersect set2;
```

dove: l'espressione contiene gli operatori **unite** e **intersect**, variabili e insiemi costanti; l'intersezione ha precedenza sull'unione; e *sono permesse* parentesi

- il programma consiste in dichiarazioni di tipo, seguite da dichiarazioni di variabile e infine da statement, come

```
typedef alpha1 over alphabet;  
typedef beta1 over { 'a', 'A', '-1', 'b', '34' };  
typedef alpha_num over alpha1 intersect beta1 unite { 'C', '9' };  
integer var1;  
alpha_num set2, set3;  
set2 = ( var1 unite { 'a', '1', 'D', '+5' } ) intersect set2;
```

dov'è d'obbligo dichiarare almeno una variabile e avere almeno uno statement

Si scriva una grammatica G in forma EBNF, non ambigua, che genera il linguaggio così tratteggiato.

Soluzione

Ecco la grammatica G richiesta (assioma PROGRAM):

G	$\langle \text{PROGRAM} \rangle \rightarrow \langle \text{TYPE_SECT} \rangle \langle \text{VAR_SECT} \rangle \langle \text{EXEC_SECT} \rangle$
	$\langle \text{TYPE_SECT} \rangle \rightarrow (\langle \text{TYPE_DECL} \rangle \text{' ;' })^*$
	$\langle \text{VAR_SECT} \rangle \rightarrow (\langle \text{VAR_DECL} \rangle \text{' ;' })^+$
	$\langle \text{EXEC_SECT} \rangle \rightarrow (\langle \text{STATEMENT} \rangle \text{' ;' })^+$
	$\langle \text{TYPE_DECL} \rangle \rightarrow \text{'typedef'} \langle \text{TYPE_ID} \rangle \text{'over'} \langle \text{SIMP_EXP} \rangle$
	$\langle \text{VAR_DECL} \rangle \rightarrow \langle \text{TYPE_ID} \rangle \langle \text{VAR_ID} \rangle (\text{' ,'} \langle \text{VAR_ID} \rangle)^*$
	$\langle \text{STATEMENT} \rangle \rightarrow \langle \text{VAR_ID} \rangle \text{'='} \langle \text{FULL_EXP} \rangle$
	$\langle \text{SIMP_EXP} \rangle \rightarrow \langle \text{SIMP_TERM} \rangle (\text{'unite'} \langle \text{SIMP_TERM} \rangle)^*$
	$\langle \text{SIMP_TERM} \rangle \rightarrow \langle \text{SIMP_FACT} \rangle (\text{'intersect'} \langle \text{SIMP_FACT} \rangle)^*$
	$\langle \text{SIMP_FACT} \rangle \rightarrow \langle \text{TYPE_ID} \rangle \mid \langle \text{SET} \rangle$
	$\langle \text{FULL_EXP} \rangle \rightarrow \langle \text{FULL_TERM} \rangle (\text{'unite'} \langle \text{FULL_TERM} \rangle)^*$
	$\langle \text{FULL_TERM} \rangle \rightarrow \langle \text{FULL_FACT} \rangle (\text{'intersect'} \langle \text{FULL_FACT} \rangle)^*$
	$\langle \text{FULL_FACT} \rangle \rightarrow \langle \text{VAR_ID} \rangle \mid \langle \text{SET} \rangle \mid (\langle \text{FULL_EXP} \rangle)$
	$\langle \text{TYPE_ID} \rangle \rightarrow \text{'alphabet'} \mid \text{'integer'} \mid \langle \text{IDENTIFIER} \rangle$
	$\langle \text{VAR_ID} \rangle \rightarrow \langle \text{IDENTIFIER} \rangle$
	$\langle \text{SET} \rangle \rightarrow \text{'{' } \langle \text{LIST} \rangle \text{'}'}$
	$\langle \text{LIST} \rangle \rightarrow \langle \text{ELEMENT} \rangle (\text{' ,'} \langle \text{ELEMENT} \rangle)^*$
	$\langle \text{ELEMENT} \rangle \rightarrow \text{' ' } (\langle \text{CHAR} \rangle \mid \langle \text{INTEGER} \rangle) \text{' '}$
	$\langle \text{IDENTIFIER} \rangle \rightarrow \langle \text{CHAR} \rangle (\langle \text{CHAR} \rangle \mid \langle \text{DIGIT} \rangle \mid \text{'_'})^*$
	$\langle \text{INTEGER} \rangle \rightarrow [\text{'+'} \mid \text{'-'}] \langle \text{DIGIT} \rangle^+$
	$\langle \text{CHAR} \rangle \rightarrow [\text{'A'} - \text{'Z'}] \mid [\text{'a'} - \text{'z'}]$
	$\langle \text{DIGIT} \rangle \rightarrow [\text{'0'} - \text{'9'}]$

La grammatica G è in forma estesa ed essendo costruita combinando modularmente strutture non ambigue, non è ambigua. Beninteso ci sono altre formulazioni, più o meno simili, e i linguaggi generati possono differire per aspetti sintattici minori.

3 Analisi sintattica e parsificatori 20%

1. È data la grammatica estesa G seguente, di alfabeto $\{c, d, e\}$ (assioma S):

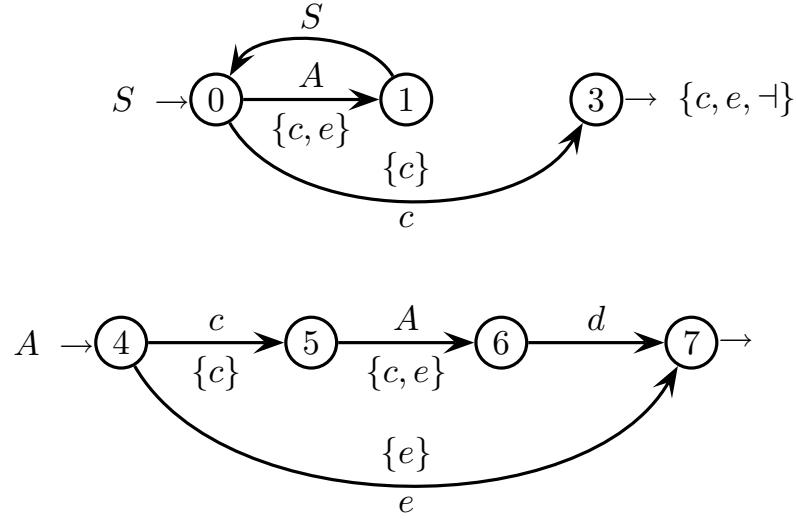
$$G \left\{ \begin{array}{l} S \rightarrow (A S)^* c \\ A \rightarrow c A d \mid e \end{array} \right.$$

Si risponda alle domande seguenti:

- (a) Si disegnino le macchine corrispondenti alle regole della grammatica G e, mostrando su di esse gl'insiemi guida rilevanti, si dica se G è $LL(1)$.
 - (b) In caso negativo, si esamini se il conflitto sia superabile con una delle tecniche seguenti:
 - allungando la lunghezza k della prospezione, ossia passando da $LL(1)$ a $LL(k)$, per $k \geq 2$
 - scrivendo una grammatica equivalente dotata della proprietà $LL(1)$
-

Soluzione

- (a) Per $k = 1$ gl'insiemi guida, riportati nelle figure seguenti, mostrano che lo stato 0 viola la condizione $LL(1)$.



La causa della violazione è il terminale di prospezione c , comune ai due archi uscenti dallo stato 0.

- (b) Non è difficile verificare che la sovrapposizione tra gl'insiemi guida degli archi uscenti dallo stato 0 persiste passando a $k = 2$:

$$Gui_2(0 \rightarrow 1) = \{ cc, ce, ec, ee \}$$

$$Gui_2(0 \rightarrow 3) = \{ cc, ce, c+ \}$$

Pertanto la grammatica G non è $LL(2)$ e la risposta non cambierebbe neppure prendendo $k > 2$.

Per rispondere alla domanda se il linguaggio L ammetta una grammatica $LL(1)$, si deve studiare il linguaggio stesso. Il linguaggio generato dal nonterminale A è $L(A) = \{ c^n e d^n \mid n \geq 0 \}$. Si noti che tali stringhe sono necessariamente generate mediante regole ricorsive e autoinclusive.

Tra le frasi del linguaggio L si trovano in particolare quelle prodotte dalle derivazioni seguenti:

$$S \Rightarrow A S c \mid c$$

ovvero le frasi del tipo seguente:

$$A^p c c^p \quad p \geq 1$$

D'altra parte, sviluppando la stella un numero arbitrario di volte, si ottengono le frasi del tipo seguente:

$$S \Rightarrow A S A S \dots A S c \overset{+}{\Rightarrow} A c A c \dots A c c$$

Si consideri ora la combinazione dei due tipi di frase. Si guardino le scelte possibili nel punto sotto evidenziato:

$$A \downarrow c A c \dots A c c$$

$$A \underset{\uparrow}{.} A^p c c^p c \quad p \geq 1$$

I primi k caratteri delle stringhe derivabili nei due casi possono coincidere, come mostrato sotto:

$$A \underset{\uparrow}{.} \underbrace{c A c \dots A c}_{c c \dots c} c$$

$$A \underset{\uparrow}{.} \underbrace{A^p}_{c c \dots c} c c^p c \quad p \geq 1$$

Dunque il linguaggio non è $LL(k)$.

2. Si consideri la grammatica G seguente, di alfabeto unario $\{a\}$ (assioma S):

$$G \left\{ \begin{array}{lcl} S & \rightarrow & a a S \\ S & \rightarrow & S a a a \\ S & \rightarrow & a \\ S & \rightarrow & \varepsilon \end{array} \right.$$

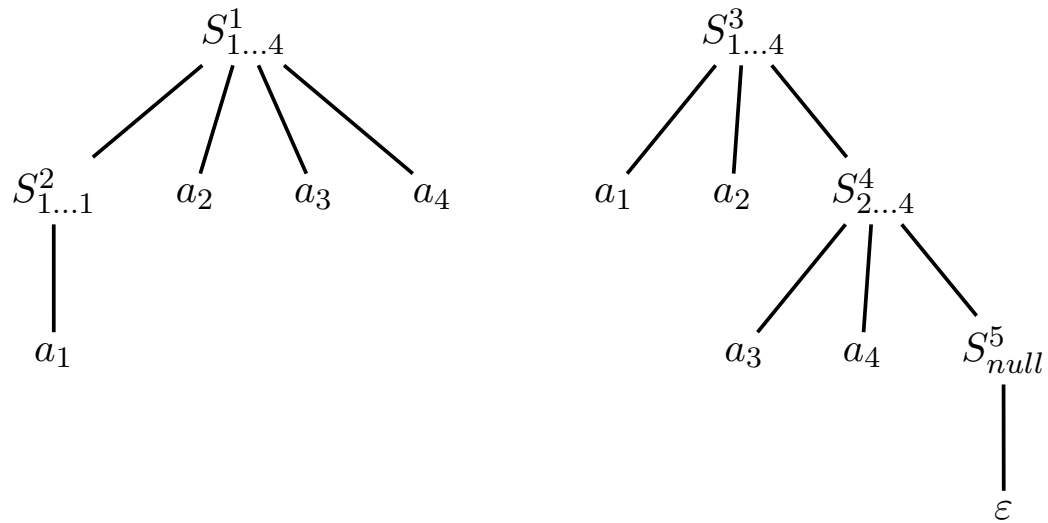
Si risponda alle domande seguenti:

- (a) Si analizzi la stringa $a a a a$ (ossia a^4) tramite il metodo di Earley, compilando la tabella a pagina seguente.
 - (b) (facoltativa) Si disegnino gli alberi sintattici della stringa, ricavabili dall'analisi.
-

[illegible]

Soluzione

- (a) Per l'analisi della stringa, vedi a pagina seguente.
 (b) Ecco i due alberi sintattici ricostruibili:



Come d'uso i nodi foglia sono numerati secondo la posizione nella stringa, mentre i nodi interni sono numerati con l'apice per metterli in corrispondenza con le candidate di riduzione nella simulazione e con il pedice per indicare la posizione iniziale e quella finale del fattore generato.

La grammatica G è ambigua. Dato che l'alfabeto terminale è unario, il linguaggio $L(G)$ è regolare. Si vede facilmente che $L(G) = a^*$.

Schema di simulazione dell'algoritmo di Earley									
stato 0	pos. a	stato 1	pos. a	stato 2	pos. a	stato 3	pos. a	stato 4	fine
$S \rightarrow \bullet a a S$	0	$S \rightarrow a \bullet a S$	0	$S \rightarrow a a \bullet S$	0	$S \rightarrow S a a a \bullet$	0	$S^1 \rightarrow S a a a \bullet$	0
$S \rightarrow \bullet S a a a$	0	$S^2 \rightarrow a \bullet$	0	$S \rightarrow S a a \bullet a$	0	$S \rightarrow S a a \bullet a$	0	$S \rightarrow a a \bullet S$	2
$S \rightarrow \bullet a$	0	$S \rightarrow S a \bullet a a$	0	$S \rightarrow S a \bullet a a$	0	$S \rightarrow a \bullet a S$	2	$S \rightarrow S a a \bullet a$	2
$S \rightarrow \bullet \varepsilon = \varepsilon \bullet$	0	$S \rightarrow S \bullet a a a$	0	$S \rightarrow \bullet a a S$	2	$S \rightarrow a \bullet$	2	$S \rightarrow S a a \bullet a$	0
$S \rightarrow S \bullet a a a$	0			$S \rightarrow \bullet S a a a$	2	$S \rightarrow S a \bullet a a$	2	$S \rightarrow S a \bullet a a$	0
				$S \rightarrow \bullet a$	2	$S \rightarrow S a \bullet a a$	0	$S \rightarrow S \bullet a a a$	0
				$S \rightarrow \bullet \varepsilon = \varepsilon \bullet$	2	$S \rightarrow S \bullet a a a$	0	$S \rightarrow \bullet a a S$	4
				$S \rightarrow a a S \bullet$	0	$S \rightarrow a a S \bullet$	0	$S \rightarrow \bullet S a a a$	4
				$S \rightarrow S \bullet a a a$	2			$S \rightarrow \bullet a$	4
				$S \rightarrow S \bullet a a a$	0			$S^5 \rightarrow \bullet \varepsilon = \varepsilon \bullet$	4
								$S^4 \rightarrow a a S \bullet$	2
								$S \rightarrow S \bullet a a a$	4
								$S^3 \rightarrow a a S \bullet$	0
								$S \rightarrow S \bullet a a a$	2

Qui sono mostrate tutte le candidate, di completamento, spostamento e riduzione. Pertanto figurano pure candidate prive di ruolo nell'eventuale generazione della stringa, come per esempio tutte quelle di spostamento che a destra del marcatore ' $\bullet$ ' contengono un numero di lettere a maggiore della lunghezza della parte di stringa che resta da analizzare. Chiaramente candidate siffatte si potrebbero omettere a priori, abbreviando parecchio la compilazione della tabella.

4 Traduzione e analisi semantica 20%

1. È dato il linguaggio sorgente L_s di alfabeto $\{a, -\}$ definito dalla e.r. seguente:

$$L_s = a (- a)^+$$

Si consideri la traduzione τ ottenuta introducendo in ogni frase di L_s *uno, e un solo, errore*, che può essere:

- sostituzione di una lettera ' a ' con il segno ' $-$ '
- sostituzione di un segno ' $-$ ' con la lettera ' a '

Incidentalmente, il linguaggio sorgente L_s coincide con il linguaggio L del problema 1.2, e il linguaggio destinazione L_d prodotto dalla traduzione τ coincide con il linguaggio L_{err} del medesimo problema.

Ecco alcuni esempi di coppie di stringhe appartenenti alla relazione di traduzione:

stringa sorgente	stringa destinazione
$a - a$	$- - a$
$a - a$	$a a a$
$a - a$	$a - -$
$a - a - a$	$- - a - a$
$\dots$	$\dots$
$a - a - a$	$a - a - -$

Si risponda alle domande seguenti:

- (a) Si scriva l'espressione regolare della traduzione τ e si verifichi quanto segue:
 - se la traduzione τ è deterministica
 - se la traduzione inversa τ^{-1} è a un solo valore
- (b) (facoltativa) Si modifichi la definizione della traduzione τ considerando, accanto ai due tipi di errori precedenti, l'errore che sta nel cancellare una, e una sola, lettera a . Nella traduzione σ così definita si hanno ora, oltre alle coppie precedenti, p. es. le coppie seguenti:

stringa sorgente	stringa destinazione
$a - a$	$-a$
$a - a$	$a-$
$a - a - a$	$-a - a$

Si scriva l'espressione regolare della traduzione σ e si dica se la traduzione inversa è una funzione a un solo valore.

Soluzione

- (a) L'espressione regolare della traduzione τ è la seguente:

$$\frac{a}{a} \left(\frac{-a}{-a} \right)^* \quad \frac{-}{a} \frac{a}{a} \left(\frac{-}{-} \frac{a}{a} \right)^* \quad | \quad \left(\frac{a-}{a-} \right)^* \quad \frac{a}{-} \left(\frac{-a}{-a} \right)^+$$

Gli esempi mostrano che una stessa stringa sorgente ha più immagini, di conseguenza la relazione τ non è una funzione e la traduzione non è deterministica.

La traduzione inversa τ^{-1} è a un solo valore per la ragione seguente: per ogni intero dispari $n \geq 3$, esiste una e una sola frase $x \in L_s$ di lunghezza n , e tutte le stringhe in $\tau(x)$ hanno la stessa lunghezza di x .

- (b) L'espressione regolare della traduzione σ è facilmente ottenibile unendo alla precedente il termine seguente:

$$\left(\frac{a-}{a-} \right)^* \quad \frac{a}{\varepsilon} \left(\frac{-}{-} \frac{a}{a} \right)^*$$

o meglio, poiché almeno un segno meno dev'esserci, il termine seguente:

$$\left(\frac{a-}{a-} \right)^+ \quad \frac{a}{\varepsilon} \left(\frac{-}{-} \frac{a}{a} \right)^* \quad | \quad \left(\frac{a-}{a-} \right)^* \quad \frac{a}{\varepsilon} \left(\frac{-}{-} \frac{a}{a} \right)^+$$

La traduzione inversa σ^{-1} resta una funzione a un solo valore.

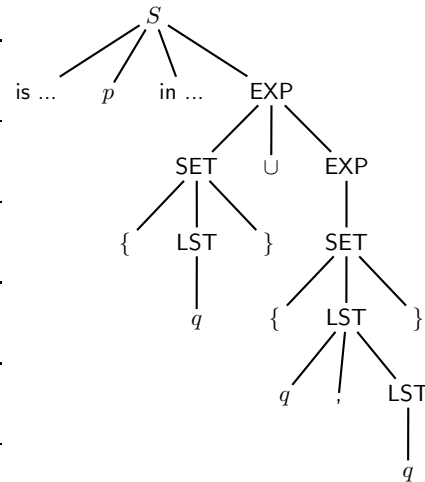
2. Si consideri il problema di cercare un elemento in un'espressione di unione finita d'insiemi finiti. Tale problema è formulato dando una clausola di ricerca, come segue:

is there element e in expression $\{a, b\} \cup \{b, c, e\} \cup \{a, e, d\}$

Qui chiaramente la risoluzione della clausola dà risposta “yes”.

Ecco il supporto sintattico G che genera la clausola di ricerca (assioma S), con a lato un piccolo albero sintattico d'esempio:

1:	$S \rightarrow \text{'is ...'} p \text{'in ...'} \langle \text{EXP} \rangle$
2:	$\langle \text{EXP} \rangle \rightarrow \langle \text{SET} \rangle \text{'\cup'} \langle \text{EXP} \rangle$
3:	$\langle \text{EXP} \rangle \rightarrow \langle \text{SET} \rangle$
4:	$\langle \text{SET} \rangle \rightarrow \text{'\{' } \langle \text{LST} \rangle \text{'\}'}$
5:	$\langle \text{LST} \rangle \rightarrow q \text{'\,' } \langle \text{LST} \rangle$
6:	$\langle \text{LST} \rangle \rightarrow q$



Per semplicità l'elemento da ricercare e tutti gli elementi dell'espressione sono schematizzati con i terminali p e q , rispettivamente, che non vanno espansi ulteriormente.

Si vuole progettare una grammatica con attributi G che risolve la clausola di ricerca, “yes” o “no” ovvero “true” o “false”, insomma che valuta un attributo *sinistro* σ (*solution*), di tipo booleano e associato all'assioma S (e se serve ad altri nonterminali). Si supponga inoltre di avere un attributo *destro* π (*propagate*), di tipo elemento e associato al nonterminale EXP (e se serve ad altri nonterminali), che propaga il valore di p dalla regola assiomatica (1) verso il resto dell'albero, dove serve. Si progettino gli attributi rimanenti (sinistri o destri) che si ritengono necessari od opportuni.

Si risponda alle domande seguenti:

- Si completi la tabella di specifica degli attributi, a pagina seguente.
- Si completi la tabella di specifica delle funzioni semantiche, a pagina seguente.
- (facoltativa) Si verifichi se la grammatica con attributi G progettata è a una scansione (one-sweep).

attributi da usare per la grammatica

tipo	nome	(non)terminali	dominio	significato
sin	σ	S	booleano	soluzione della clausola
des	π	EXP	elemento	valore dell'elemento p da propagare per la ricerca nell'espressione

sintassi

semantica

$S_0 \rightarrow \text{'is ...'} p \text{'in ...'} \langle \text{EXP} \rangle_1$	$\pi_1 = p$ $\sigma_0 =$
$\langle \text{EXP} \rangle_0 \rightarrow \langle \text{SET} \rangle_1 \text{'}\cup\text{' } \langle \text{EXP} \rangle_2$	$\pi_2 = \pi_0$
$\langle \text{EXP} \rangle_0 \rightarrow \langle \text{SET} \rangle_1$	
$\langle \text{SET} \rangle_0 \rightarrow \text{'}\{\text{' } \langle \text{LST} \rangle_1 \text{'}\}$	
$\langle \text{LST} \rangle_0 \rightarrow q \text{'},\text{' } \langle \text{LST} \rangle_1$	
$\langle \text{LST} \rangle_0 \rightarrow q$	

Soluzione

(a) Ecco gli attributi completati e quelli nuovi.

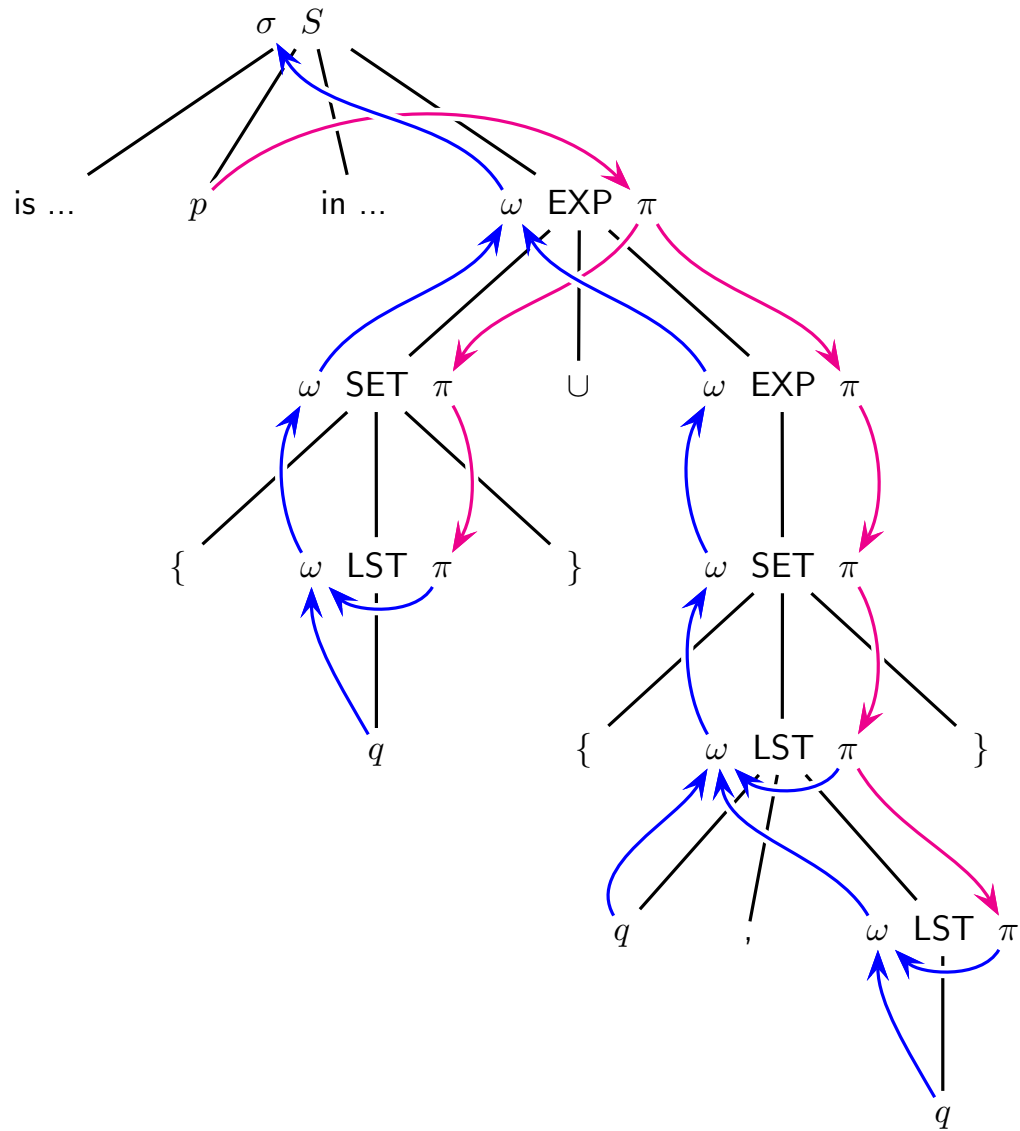
attributi da usare per la grammatica				
tipo	nome	(non)terminali	dominio	significato
sin	σ	S	booleano	soluzione della clausola
des	π	EXP, SET, LST	elemento	valore dell'elemento p da propagare per la ricerca nell'espressione
sin	ω	EXP, SET, LST	booleano	predicato di appartenenza dell'elemento p alla (sotto)espressione

(b) Ecco le funzioni semantiche completate e quelle nuove.

sintassi	semantica
$S_0 \rightarrow \text{'is ...'} p \text{'in ...'} \langle \text{EXP} \rangle_1$	$\pi_1 = p$ $\sigma_0 = \omega_1$
$\langle \text{EXP} \rangle_0 \rightarrow \langle \text{SET} \rangle_1 \text{'\(\cup\')} \langle \text{EXP} \rangle_2$	$\pi_2 = \pi_0$ $\pi_1 = \pi_0$ $\omega_0 = \omega_1 \vee \omega_2$
$\langle \text{EXP} \rangle_0 \rightarrow \langle \text{SET} \rangle_1$	$\pi_1 = \pi_0$ $\omega_0 = \omega_1$
$\langle \text{SET} \rangle_0 \rightarrow \text{'\{'} \langle \text{LST} \rangle_1 \text{'\}'}$	$\pi_1 = \pi_0$ $\omega_0 = \omega_1$
$\langle \text{LST} \rangle_0 \rightarrow q \text{'\,'} \langle \text{LST} \rangle_1$	$\pi_1 = \pi_0$ $\omega_0 = (\pi_0 == q) \vee \omega_1$
$\langle \text{LST} \rangle_0 \rightarrow q$	$\omega_0 = (\pi_0 == q)$

(c) La grammatica G è a una scansione (one-sweep), perché si risolve nello scendere lungo l'albero propagando l'attributo destro (ereditato) π e nel risalire valutando l'attributo sinistro (sintetizzato) ω , fino a calcolare l'attributo σ nella radice.

Per completezza, ecco il grafo delle dipendenze per l'albero sintattico d'esempio (non richiesto):



Da questo albero si vede bene che la grammatica G è a scansione e che non occorre neppure cambiare l'ordinamento da sinistra verso destra dei nodi figli.